

자동차IT융합학과 20241268 지윤주
자동차인공지능 이상헌교수님

2026/05/08

HW 4A

MNIST 숫자 분류 신경망의
하이퍼파라미터 비교
_PYTHON

1. 서론

1.1. 모델 구조

본 실험에서 사용한 신경망은 입력층부터 출력층까지 총 5개의 층으로 구성된 Multi-Layer Perceptron 구조이다. 입력층에는 28*28 크기의 MNIST 이미지를 1차원 벡터로 평탄화하여 사용하였다. 4개의 Hidden nodes를 거치며 노드 수가 점진적으로 축소된다. 각 은닉층에서는 ReLU의 활성화 함수를 배치하였고, 최종적으로 Softmax가 포함된 Cross Entropy Loss를 사용하였다.

1.2. 실험 조건

- 데이터셋 : MNIST 숫자 데이터셋 (Training : 60,000장 / Validation : 10,000장 제외)

- 통제 변인

Optimizer : SGD

Initialization : Xavier

Regularization : Dropout = 0

Learning Rate : 0.01

Batch size : 100

Epochs : 20

set_seed(42) # 시드 고정

- 조작 변인

Optimizer : SGD / Adam

Initialization : Xavier / He

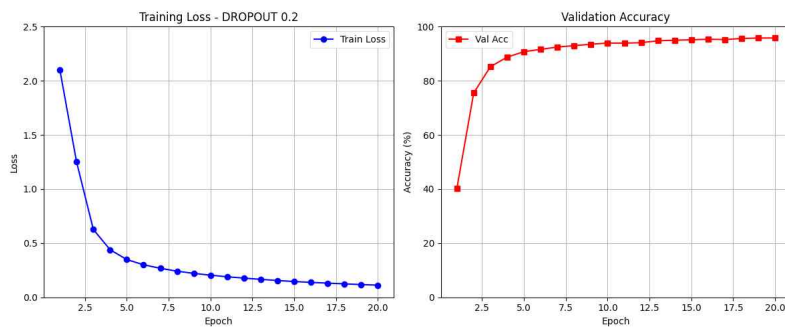
Regularization : Dropout=0/ 0.5 (또는 0.2)

2. 본론 - 실험 결과 및 분석

2.1. 실험 결과 표

Experiment	Setting	Train Acc	Val Acc	Test Acc	Comment
Optimizer	SGD	97.70	95.82	96.03	
Optimizer	Adam	99.24	97.22	96.97	성능, 속도 향상
Initialization	Xavier	97.70	95.82	96.03	
Initialization	He	98.18	95.94	96.24	미세하게 향상
Dropout	없음	97.70	95.82	96.03	
Dropout	0.2	96.76	94.71	94.92	과도 규제

2.2. Baseline Acc 추적 (SGD + Xavier + Dropout=0)

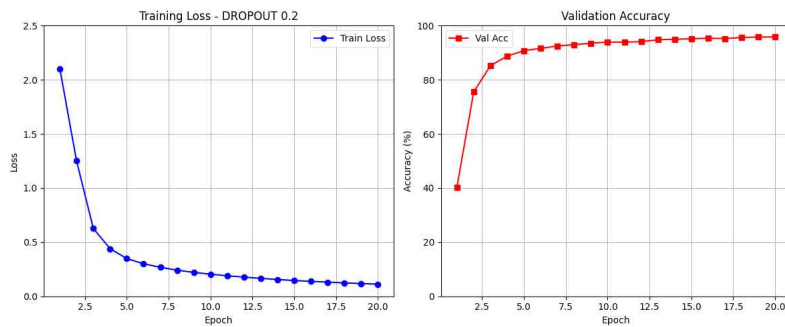


```
print('Epoch: {epoch}/7 (EPOCHS) Loss: {avg_loss:.4f}, Train
```

Epoch [1/20]	Loss: 2.1081	Train Acc: 28.67%	Val Acc: 40.35%
Epoch [2/20]	Loss: 1.2506	Train Acc: 62.14%	Val Acc: 75.50%
Epoch [3/20]	Loss: 0.6273	Train Acc: 81.15%	Val Acc: 85.22%
Epoch [4/20]	Loss: 0.4401	Train Acc: 88.53%	Val Acc: 88.73%
Epoch [5/20]	Loss: 0.3488	Train Acc: 91.00%	Val Acc: 90.76%
Epoch [6/20]	Loss: 0.3008	Train Acc: 92.25%	Val Acc: 91.60%
Epoch [7/20]	Loss: 0.2680	Train Acc: 93.24%	Val Acc: 92.48%
Epoch [8/20]	Loss: 0.2418	Train Acc: 93.93%	Val Acc: 92.97%
Epoch [9/20]	Loss: 0.2212	Train Acc: 94.49%	Val Acc: 93.50%
Epoch [10/20]	Loss: 0.2046	Train Acc: 95.03%	Val Acc: 93.95%
Epoch [11/20]	Loss: 0.1897	Train Acc: 95.48%	Val Acc: 93.91%
Epoch [12/20]	Loss: 0.1775	Train Acc: 95.87%	Val Acc: 94.07%
Epoch [13/20]	Loss: 0.1660	Train Acc: 96.11%	Val Acc: 94.78%
Epoch [14/20]	Loss: 0.1555	Train Acc: 96.40%	Val Acc: 94.97%
Epoch [15/20]	Loss: 0.1459	Train Acc: 96.68%	Val Acc: 95.15%
Epoch [16/20]	Loss: 0.1381	Train Acc: 96.96%	Val Acc: 95.35%
Epoch [17/20]	Loss: 0.1311	Train Acc: 97.17%	Val Acc: 95.23%
Epoch [18/20]	Loss: 0.1244	Train Acc: 97.35%	Val Acc: 95.62%
Epoch [19/20]	Loss: 0.1179	Train Acc: 97.53%	Val Acc: 95.78%
Epoch [20/20]	Loss: 0.1126	Train Acc: 97.70%	Val Acc: 95.82%

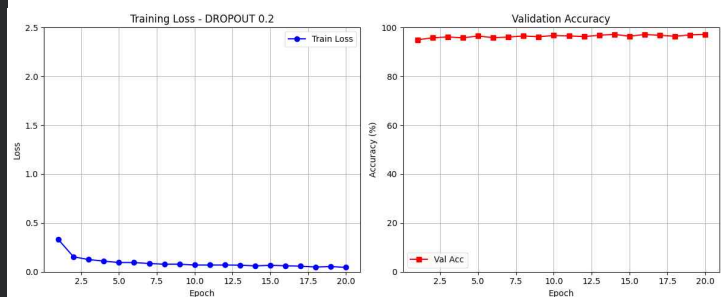
2.2. Optimizer 비교

SGD



ADAM

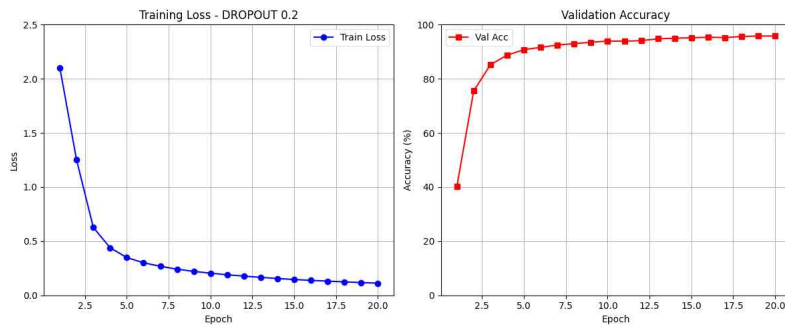
Epoch [1/20]	Loss: 0.3300, Train Acc: 91.69%, Val Acc: 94.98%
Epoch [2/20]	Loss: 0.1533, Train Acc: 96.82%, Val Acc: 95.81%
Epoch [3/20]	Loss: 0.1254, Train Acc: 97.52%, Val Acc: 96.14%
Epoch [4/20]	Loss: 0.1096, Train Acc: 97.93%, Val Acc: 96.80%
Epoch [5/20]	Loss: 0.0950, Train Acc: 98.28%, Val Acc: 96.51%
Epoch [6/20]	Loss: 0.0946, Train Acc: 98.26%, Val Acc: 95.87%
Epoch [7/20]	Loss: 0.0853, Train Acc: 98.45%, Val Acc: 96.09%
Epoch [8/20]	Loss: 0.0776, Train Acc: 98.62%, Val Acc: 96.55%
Epoch [9/20]	Loss: 0.0780, Train Acc: 98.60%, Val Acc: 96.21%
Epoch [10/20]	Loss: 0.0695, Train Acc: 98.71%, Val Acc: 96.75%
Epoch [11/20]	Loss: 0.0694, Train Acc: 98.78%, Val Acc: 96.58%
Epoch [12/20]	Loss: 0.0696, Train Acc: 98.77%, Val Acc: 96.27%
Epoch [13/20]	Loss: 0.0682, Train Acc: 98.78%, Val Acc: 96.87%
Epoch [14/20]	Loss: 0.0604, Train Acc: 98.90%, Val Acc: 97.16%
Epoch [15/20]	Loss: 0.0665, Train Acc: 98.93%, Val Acc: 96.46%
Epoch [16/20]	Loss: 0.0618, Train Acc: 98.90%, Val Acc: 97.13%
Epoch [17/20]	Loss: 0.0576, Train Acc: 99.02%, Val Acc: 96.83%
Epoch [18/20]	Loss: 0.0491, Train Acc: 99.13%, Val Acc: 96.44%
Epoch [19/20]	Loss: 0.0540, Train Acc: 99.09%, Val Acc: 96.98%
Epoch [20/20]	Loss: 0.0450, Train Acc: 99.24%, Val Acc: 97.22%



- 어느 optimizer가 더 빠르게 수렴하는가? 어느 optimizer가 더 안정적인가?
ADAM이 SGD보다 훨씬 빠르고 안정적으로 수렴한다.

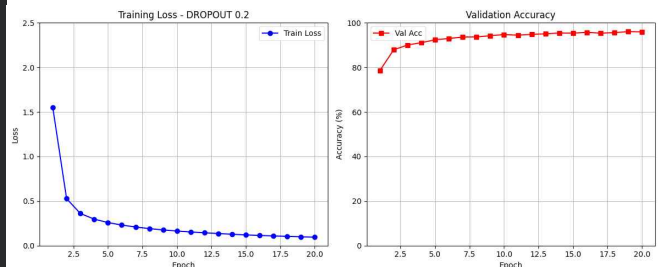
2.3. Initialization 비교

XAVIER



HE

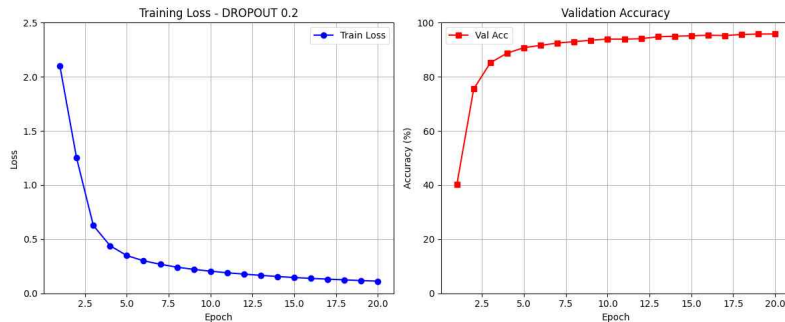
```
** Epoch [1/20] Loss: 1.5473, Train Acc: 51.62%, Val Acc: 78.58%
Epoch [2/20] Loss: 0.5271, Train Acc: 86.18%, Val Acc: 87.90%
Epoch [3/20] Loss: 0.3608, Train Acc: 90.68%, Val Acc: 90.04%
Epoch [4/20] Loss: 0.2974, Train Acc: 92.50%, Val Acc: 91.07%
Epoch [5/20] Loss: 0.2592, Train Acc: 93.57%, Val Acc: 92.40%
Epoch [6/20] Loss: 0.2307, Train Acc: 94.23%, Val Acc: 92.96%
Epoch [7/20] Loss: 0.2094, Train Acc: 94.89%, Val Acc: 93.61%
Epoch [8/20] Loss: 0.1909, Train Acc: 95.40%, Val Acc: 93.69%
Epoch [9/20] Loss: 0.1763, Train Acc: 95.80%, Val Acc: 94.20%
Epoch [10/20] Loss: 0.1641, Train Acc: 96.17%, Val Acc: 94.73%
Epoch [11/20] Loss: 0.1533, Train Acc: 96.45%, Val Acc: 94.46%
Epoch [12/20] Loss: 0.1445, Train Acc: 96.72%, Val Acc: 94.82%
Epoch [13/20] Loss: 0.1356, Train Acc: 97.01%, Val Acc: 95.05%
Epoch [14/20] Loss: 0.1279, Train Acc: 97.21%, Val Acc: 95.43%
Epoch [15/20] Loss: 0.1205, Train Acc: 97.43%, Val Acc: 95.35%
Epoch [16/20] Loss: 0.1146, Train Acc: 97.60%, Val Acc: 95.72%
Epoch [17/20] Loss: 0.1090, Train Acc: 97.74%, Val Acc: 95.35%
Epoch [18/20] Loss: 0.1044, Train Acc: 97.93%, Val Acc: 95.60%
Epoch [19/20] Loss: 0.0991, Train Acc: 98.04%, Val Acc: 95.05%
Epoch [20/20] Loss: 0.0951, Train Acc: 98.18%, Val Acc: 95.94%
```



- 초기화 방법에 따라 학습 초반의 안정성이 어떻게 달라졌는가?
 - ReLU를 사용할 때 어떤 초기화가 더 적절하였는가?
- ReLU에서 분산이 작아지는 것을 보정해 주는 He가 미세하게 더 안정적인 성능을 보인다.

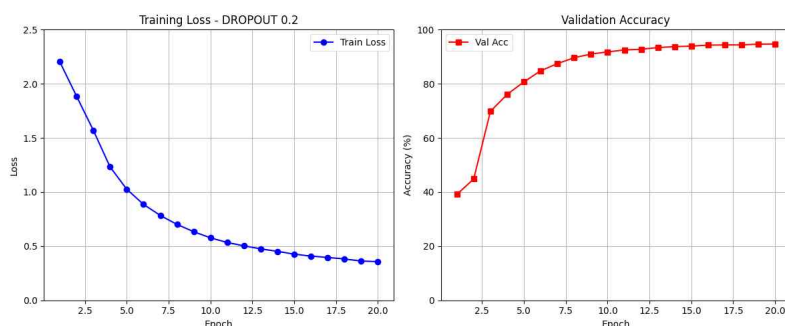
2.4. Dropout 비교

DROPOUT 0



dropout 0.2

```
Epoch [1/20] Loss: 2.2026, Train Acc: 28.40%, Val Acc: 39.17%
Epoch [2/20] Loss: 1.8850, Train Acc: 40.49%, Val Acc: 44.83%
Epoch [3/20] Loss: 1.5707, Train Acc: 59.55%, Val Acc: 69.81%
Epoch [4/20] Loss: 1.2325, Train Acc: 73.36%, Val Acc: 76.09%
Epoch [5/20] Loss: 1.0257, Train Acc: 79.26%, Val Acc: 80.69%
Epoch [6/20] Loss: 0.8859, Train Acc: 83.68%, Val Acc: 84.82%
Epoch [7/20] Loss: 0.7842, Train Acc: 86.72%, Val Acc: 87.44%
Epoch [8/20] Loss: 0.7009, Train Acc: 89.41%, Val Acc: 89.63%
Epoch [9/20] Loss: 0.6337, Train Acc: 91.07%, Val Acc: 90.98%
Epoch [10/20] Loss: 0.5770, Train Acc: 91.98%, Val Acc: 91.78%
Epoch [11/20] Loss: 0.5343, Train Acc: 92.66%, Val Acc: 92.50%
Epoch [12/20] Loss: 0.5029, Train Acc: 93.20%, Val Acc: 92.75%
Epoch [13/20] Loss: 0.4757, Train Acc: 93.71%, Val Acc: 93.39%
Epoch [14/20] Loss: 0.4519, Train Acc: 94.08%, Val Acc: 93.70%
Epoch [15/20] Loss: 0.4268, Train Acc: 94.47%, Val Acc: 93.91%
Epoch [16/20] Loss: 0.4083, Train Acc: 94.79%, Val Acc: 94.28%
Epoch [17/20] Loss: 0.3960, Train Acc: 95.06%, Val Acc: 94.35%
Epoch [18/20] Loss: 0.3821, Train Acc: 95.31%, Val Acc: 94.37%
Epoch [19/20] Loss: 0.3630, Train Acc: 95.53%, Val Acc: 94.64%
Epoch [20/20] Loss: 0.3578, Train Acc: 95.76%, Val Acc: 94.71%
```



- dropout이 과적합 완화에 도움이 되었는가?
- training 성능과 validation 성능의 차이가 어떻게 변했는가?

MNIST의 데이터 특성상 과적합의 오류가 크게 일어나지 않아서 학습을 지나치게 규제하면 오히려 성능이 떨어지는 결과가 나타난다. DROPOUT의 정도를 0.2 정도로 적절히 맞췄을 때 그나마 안정적인 성능을 보인다.

3. 결론

1. HW3의 단일 은닉층 신경망과 이번 과제의 다층 신경망은 성능 면에서 어떤 차이가 있었는가?

단일 은닉층 신경망에서는 높은 정확도를 도출할 수는 있었으나, 하이퍼파라미터 변화에 따라 성능 편차가 크게 나타났다. 이번 다층 신경망에서는 모델이 깊어짐으로써 다양한 조합에서도 비교적 안정적으로 높은 성능을 유지하는 것을 확인할 수 있었다.

2. SGD와 Adam 중 어느 optimizer가 더 잘 작동하였는가?

Adam이 더 우수한 성능을 보였다. SGD는 학습률이 고정되어 있어 상대적으로 수렴이 느렸으나 Adam은 더 빠르고 안정적으로 높은 정확도에 도달했다.

3. Xavier와 He 초기화 중 어느 쪽이 더 적절하였는가?

활성화 함수로 ReLU를 사용했기 때문에 He 초기화가 미세하게 좋은 성능을 보였다. 이는 He 초기화가 가중치 분산을 조절했기 때문이다.

4. Dropout은 과적합 완화에 도움이 되었는가?

Train 과 Validation Acc 사이의 간격을 줄이기는 했으나, 단순한 데이터셋의 특성상 오히려 학습을 방해하는 요소가 되었다. 특히 $p=0.5$ 에서는 성능이 급감하여 $p=0.2$ 로 낮추어 균형 잡힌 성능을 확인했다.

5. 가장 성능이 좋았던 하이퍼파라미터 조합은 무엇인가?

Adam, He, dropout=0 의 조합이다. 이 조합에서 Val Acc의 최고 수치를 기록했다.

6. train 성능과 validation 성능 차이를 통해 과적합 또는 과소적합을 확인할 수 있었는가?

Dropout=0에서 전반저금로 Train Acc 가 Val Acc보다 높게 유지되는 과적합을 미세하게 확인할 수 있었다.

7. 깊은 신경망에서 가장 중요하다고 생각되는 요소는 무엇이며, 그 이유는 무엇인가?

가중치 초기화와 규제의 균형이 가장 중요하다. 특히 강력한 규제를 바로 거는 것보다 데이터의 특성을 파악하고 적절한 파라미터를 찾아내는 것이 중요하다.